

RistoBook

Iterazione 1

1. Analisi Concettuale	3
1.1. Raffinamento dei requisiti	3
Raffinamento dei requisiti	3
Requisiti che riguardano le funzionalità	4
1.2. Diagramma UML delle classi	5
1.3. Specifiche di tipi di dato	6
1.4. Specifiche delle classi	6
1.4.1. Specifica della classe Prenotazione	6
Operazioni Ausiliarie	6
non_sovrapposti(i1: DataOra, f1: DataOra, i2: DataOra, f2: DataOra) : Booleano	6
non_sovrapposti_data(i1: Data, f1: Data, i2: Data, f2: Data) : Booleano	6
non_sovrapposti_orariogiorno(og1:OrarioGiorno, og2:OrarioGiorno): Booleano	6
Vincoli Esterni	7
[V.IstanteCreazioneMinoreDataOraPrenotata]	7
[V.IstanteCreazioneMinoreDiAccettazione]	7
[V.IstanteCreazioneMinoreDiRifiuto]	7
[V.DataOraPrenotataNonInPeriodoChiusura]	7
[v.PeriodoChiusura.NonSovrapposti]	7
[v.OrarioGiorno.NonSovrapposto]	7
1.4.2. Specifica della classe Ristorante	8
Operazioni ausiliarie	8
is_in_periodo_chiusura(i: DataOra) : Booleano	8
1.4.3. Specifiche di classe Cliente	8
num_prenot_accettate(d_iniz:DataOra, d_fine:DataOra): Intero >=0	8
num_prenot_rifiutate(d_iniz:DataOra, d_fine:DataOra): Intero >=0	8
1.4. Diagramma degli Use-Case	9
1.5. Specifiche degli Use-Case	9
1.5.1. Specifica dello use-case Registrazione a RistoBook	9
registrazione_cliente(	9
registrazione_ristorante(	10
1.5.2. Specifica dello use-case Effettua prenotazioni	10
effettuare_prenotazioni(	10
1.5.3. Specifica dello use-case Gestione prenotazioni	10
ristorante_accetta_prenotazione(p: Prenotazione) : Accettata	10
ristorante_rifiuta_prenotazione(p: Prenotazione) : Rifiutata	11
1.5.4. Specifica dello use-case Gestione ristorante	11
creazione_periodo_di_chiusura(	11
modifica_orario_giorno_in_periodo_chiusura(	11
elimina_periodo_di_chiusura(	12
aggiunta_tipo_cucina(r: Ristorante, tip: TipoCucina)	12
elimina_tipo_cucina(r: Ristorante, tip: TipoCucina)	12
1.5.5. Specifica dello use-case Statistiche	12
num_prenotazioni_ristorante_nel_periodo(	12
num_prenot_accettate_ristorante_nel_periodo(	13
num_prenot_rifiutate_ristorante_nel_periodo(	13
num_prenotazioni_effetuate_da cliente_nel_periodo(	13
num_prenotazioni_effetuate_da cliente_nel_periodo_nel_ristorante(	14

1.5.6. Specifica dello use-case Gestione Ubicazione	14
creazione_citta(nome: Stringa, reg: Regione) : Citta	14
creazione_regione(nome: Stringa, naz: Nazione) : Regione	14
creazione_nazione(nome: Stringa) : Nazione	15
1.5.7. Specifica dello use-case Gestione tipi cucina	15
creazione_tipo_cucina(nom: Stringa) : TipoCucina	15
2. Design dell'applicazione Python	15
2.1. Diagramma UML delle classi Ristrutturato	15
2.2. Specifiche delle Classi	16
2.2.1. Specifica della classe Prenotazione	16
Operazioni Ausiliarie	16
non_sovrapposti(i1: datetime, f1: datetime, i2: datetime, f2: datetime) : bool	16
non_sovrapposti_data(i1:datetime.date, f1:datetime.date, i2:datetime.date, f2: datetime.date): bool	16
non_sovrapposti_orariogiorno(og1: OrarioGiorno, og2: OrarioGiorno): bool	17
Vincoli Esterni	17
[V.IstanteCreazioneMinoreDataOraPrenotata]	17
[V.IstanteCreazioneMinoreDiAccettazione]	17
[V.IstanteCreazioneMinoreDiRifiuto]	17
[V.DataOraPrenotataNonInPeriodoChiusura]	17
[Da fusione] per ogni p:Prenotazione:	17
[v.OrarioGiorno.NonSovrapposto]	18
[v.OrarioGiorno.NonSovrapposto]	18
2.2.2. Specifica della classe Ristorante	18
Operazioni ausiliarie	18
is_in_periodo_chiusura(i: datetime) : bool	18
2.2.3. Specifiche di classe Cliente	19
num_prenot_accettate(d_iniz: datetime, d_fine: datetime): IntGEZ	19
num_prenot_rifiutate(d_iniz: datetime, d_fine: datetime): IntGEZ	19
2.3. Specifiche degli Use-Case	20
2.3.1. Specifica dello use-case Registrazione a RistoBook	20
registrazione_cliente(	20
registrazione_ristorante(	21
2.3.2. Specifica dello use-case Effettua prenotazioni	21
effettuare_prenotazioni(	21
2.3.3. Specifica dello use-case Gestione prenotazioni	22
ristorante_accetta_prenotazione(p: Prenotazione)	22
ristorante_rifiuta_prenotazione(p: Prenotazione)	23
2.3.4. Specifica dello use-case Gestione ristorante concettuale	23
creazione_periodo_di_chiusura(	23
modifica_orario_giorno_in_periodo_chiusura(	24
elimina_periodo_di_chiusura(	25
aggiunta_tipo_cucina(r: Ristorante, tip: TipoCucina)	25
elimina_tipo_cucina(r: Ristorante, tip: TipoCucina)	25
2.3.5. Specifica dello use-case Statistiche	26
num_prenotazioni_ristorante_nel_periodo(	26
num_prenot_accettate_ristorante_nel_periodo(	27
num_prenot_rifiutate_ristorante_nel_periodo(	27
num_prenotazioni_effettuate_da_cliente_nel_periodo_nel_ristorante(	29
2.3.6. Specifica dello use-case Gestione Ubicazione	29
creazione_citta(nome: str, reg: Regione) : Citta	29
creazione_regione(nome: str, naz: Nazione) : Regione	30

1. Analisi Concettuale

1.1. Raffinamento dei requisiti

Obiettivo: realizzare una prima versione funzionante di Ristobook che permette

- ai clienti di registrarsi e fare prenotazioni
- ai ristoratori di registrare i propri ristoranti, comprese le tipologie di cucina offerte e i periodi di chiusura delle prenotazioni
- uso di stati base per le prenotazioni: pendente, accettata, rifiutata

Raffinamento dei requisiti

1. Requisiti relativi ai clienti:
 - 1.1. nome
 - 1.2. email
 - 1.3. prenotazioni effettuate
2. Requisiti relativi ai ristoratori:
 - 2.1. nome
 - 2.2. ristoranti gestiti
3. Requisiti relativi ai ristoranti:
 - 3.1. nome
 - 3.2. partita iva
 - 3.3. indirizzo
 - 3.4. città
 - 3.5. tipologie di cucine offerte (anche più di una) (v. req. 6)
 - 3.7. periodo di chiusura (opzionale, anche più di uno):
 - 3.7.1. data inizio chiusura
 - 3.7.2. data fine chiusura
 - 3.7.3. giorni di settimana di chiusura
 - 3.7.3.1. fasce orarie
4. Requisiti relativi alle prenotazioni
 - 4.1. cliente che ha fatto la prenotazione (v. req. 1)
 - 4.2. ristorante scelto (v. req. 3)
 - 4.3. data e ora prenotata
 - 4.4. numero commensali
 - 4.5. istante di creazione
 - 4.6. stato di prenotazione (v. req. 5)
 - 4.7. storico dei cambiamenti di stato
 - 4.7.1. nuovo stato
 - 4.7.2. data e ora del cambiamento
 - 4.7.3. chi ha fatto la modifica (cliente o ristorante)

5. Requisiti relativi allo stato di prenotazione

5.1. stati base

5.1.1. stato "pendente" (in momento di creazione di prenotazione)

5.1.1.1. istante quando prenotazione viene creata

5.1.2. stato "rifiutata" (quando personale del ristorante rifiuta prenotazione)

5.1.2.1. istante quando prenotazione viene "rifiutata"

5.1.3. stato "accettata" (quando personale del ristorante accetta prenotazione)

5.1.3.1. istante quando prenotazione viene "accettata"

6. Requisiti relativi alla tipologia della cucina:

6.1. nome

6.2. lista gestita dallo staff di Ristobook

Requisiti che riguardano le funzionalità

999.1. I ristoratori possono iscriversi a RistoBook per registrare i propri ristoranti e gestire le loro prenotazioni

999.2. I ristoranti possono scegliere le tipologie di cucina da una lista gestita dallo staff di Ristobook

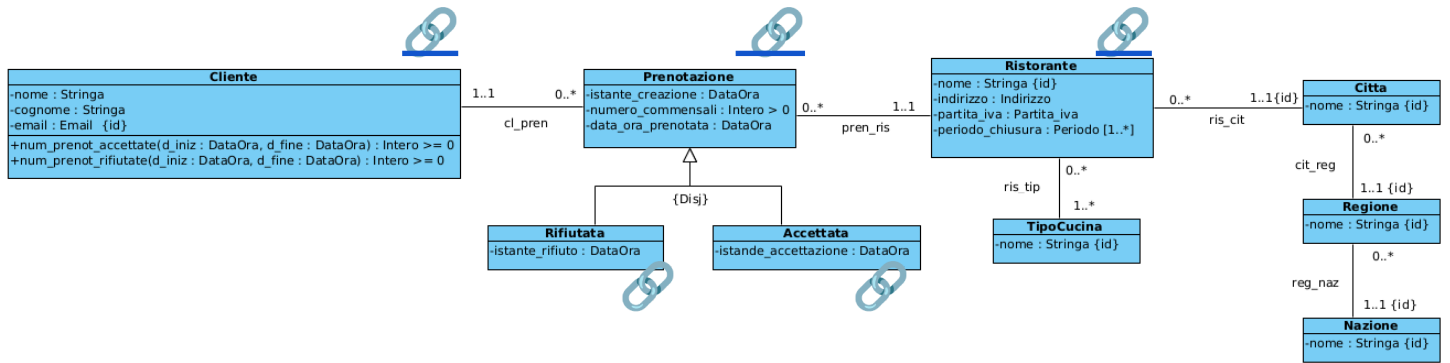
999.4. Il personale incaricato dei rispettivi ristoranti, deve poter accedere a Ristobook tramite una interfaccia dedicata

999.5. I clienti possono iscriversi a Ristobook ed effettuare prenotazioni presso i ristoranti iscritti

999.6. Il sistema deve permettere ai ristoratori di gestire lo stato delle prenotazioni

999.7. I ristoratori possono chiudere le prenotazioni per un periodo (date scelte o giorni della settimana) e fascia oraria

1.2. Diagramma UML delle classi



1.3. Specifiche di tipi di dato

- Tipo Partitalva: Stringa di 11 caratteri secondo standard
- Tipo Indirizzo: Tipo record con campi
 - via: Stringa
 - civico: intero > 0
- Tipo Email: Stringa secondo standard
- Tipo Periodo: Tipo record con campi
 - data_inizio: Data
 - data_fine: Data
 - orari: OrarioGiorno[1..*]
- Tipo OrarioGiorno: Tipo record con campi
 - giorno_settimana: {lunedì, martedì, mercoledì, giovedì, venerdì, sabato, domenica}
 - ora_inizio: Ora
 - ora_fine: Ora
- Tipo Percentuale: Tipo record con campi
 - numero: intero > 0
 - simbolo: %

1.4. Specifiche delle classi

1.4.1. Specifica della classe Prenotazione

Operazioni Ausiliarie

non_sovrapposti(i_1 : DataOra, f_1 : DataOra, i_2 : DataOra, f_2 : DataOra) : Booleano

ME-approved ▾

- precondizioni:
 - $i_1 \leq f_1$ e $i_2 \leq f_2$
- postcondizioni:
 - result = True se e solo se:
 - $i_1 > f_2$ or $f_1 < i_2$

non_sovrapposti_data(i_1 : Data, f_1 : Data, i_2 : Data, f_2 : Data) : Booleano

ME-approved ▾

- precondizioni:
 - $i_1 \leq f_1$ e $i_2 \leq f_2$
- postcondizioni:
 - result = True se e solo se:
 - $i_1 > f_2$ or $f_1 < i_2$

non_sovrapposti_orariogiorno(og1:OrarioGiorno, og2:OrarioGiorno): Booleano

ME-approved ▾

- precondizioni:
 - nessuna
- postcondizioni:
 - Result = True
 - se e solo se
 - [og1.giorno_settimana $\neq$ og2.giorno_settimana
oppure
• og1.giorno_settimana = og2.giorno_settimana
e

- NON ESISTE un i: Ora tale che
 - $og1.ora_inizio \leq i \leq og1.ora_fine$ e
 - $og2.ora_inizio \leq i \leq og2.ora_fine$
-]

Vincoli Esterni

[V.IstanteCreazioneMinoreDataOraPrenotata]

ME-approved ▾

Per ogni p:Prenotazione deve essere vero che $p.istante_creazione < p.data_ora_prenotata$

[V.IstanteCreazioneMinoreDiAccettazione]

ME-approved ▾

Per ogni p:Prenotazione deve essere vero che $p.istante_creazione < p.istante_accettazione$

[V.IstanteCreazioneMinoreDiRifiuto]

ME-approved ▾

Per ogni p:Prenotazione deve essere vero che $p.istante_creazione < p.istante_rifiuto$

[V.DataOraPrenotataNonInPeriodoChiusura]

ME-approved ▾

Per ogni p:Prenotazione con valore $i = p.data_ora_prenotata$ e (p,r): $pren_ris$, r:Ristorante.
 $r.is_in_periodo_chiusura(i) = FALSE$

[v.PeriodoChiusura.NonSovrapposti]

chiara.macciocchi17@gmail.com termina tu

ME-approved ▾

- Per ogni r:Ristorante
 - Per ogni coppia di valori distinti pc_1 e pc_2 di $r.periodo_chiusura$ deve essere vero che
 - $non_sovrapposti_data$ (
 - $pc_1.data_inizio:Data$,
 - $pc_1.data_fine:Data$,
 - $pc_2.data_inizio:Data$,
 - $pc_2.data_fine:Data$) = True

[v.OrarioGiorno.NonSovrapposto]

chiara.macciocchi17@gmail.com

ME-approved ▾

- Per ogni r:Ristorante
 - Per ogni pc:[Periodo](#) tale che pc è uno dei valori di $r.periodo_chiusura$;
 - Per ogni coppia di valori distinti $og1:OrarioGiorno$, $og2:OrarioGiorno$ di $pc.orari$, deve essere vero che
 - $non_sovrapposti_orariogiorno$ (
 - $og1:OrarioGiorno$,
 - $og2:OrarioGiorno$) = True

1.4.2. Specifica della classe [Ristorante](#)

Operazioni ausiliarie

is_in_periodo_chiusura(i: DataOra) : Booleano

ME-approved ▾

- precondizioni : nessuna
- postcondizioni:
 - $result = TRUE \Leftrightarrow [$
 - $i \notin [this.periodo_chiusura.data_inizio, this.periodo_chiusura.data_fine]$
 - oppure
 - per ogni valore og di this.periodo_chiusura.orari
 - $i.data.giorno.settimana \neq og.giorno_settimana$
 - oppure
 - $(i.data.ora \leq og.ora_inizio -) \text{ or } i.data.ora \geq og.ora_fine]$

1.4.3. Specifiche di classe [Cliente](#)

num_prenot_accettate(d_iniz:DataOra, d_fine:DataOra): Intero ≥ 0

ME-approved ▾

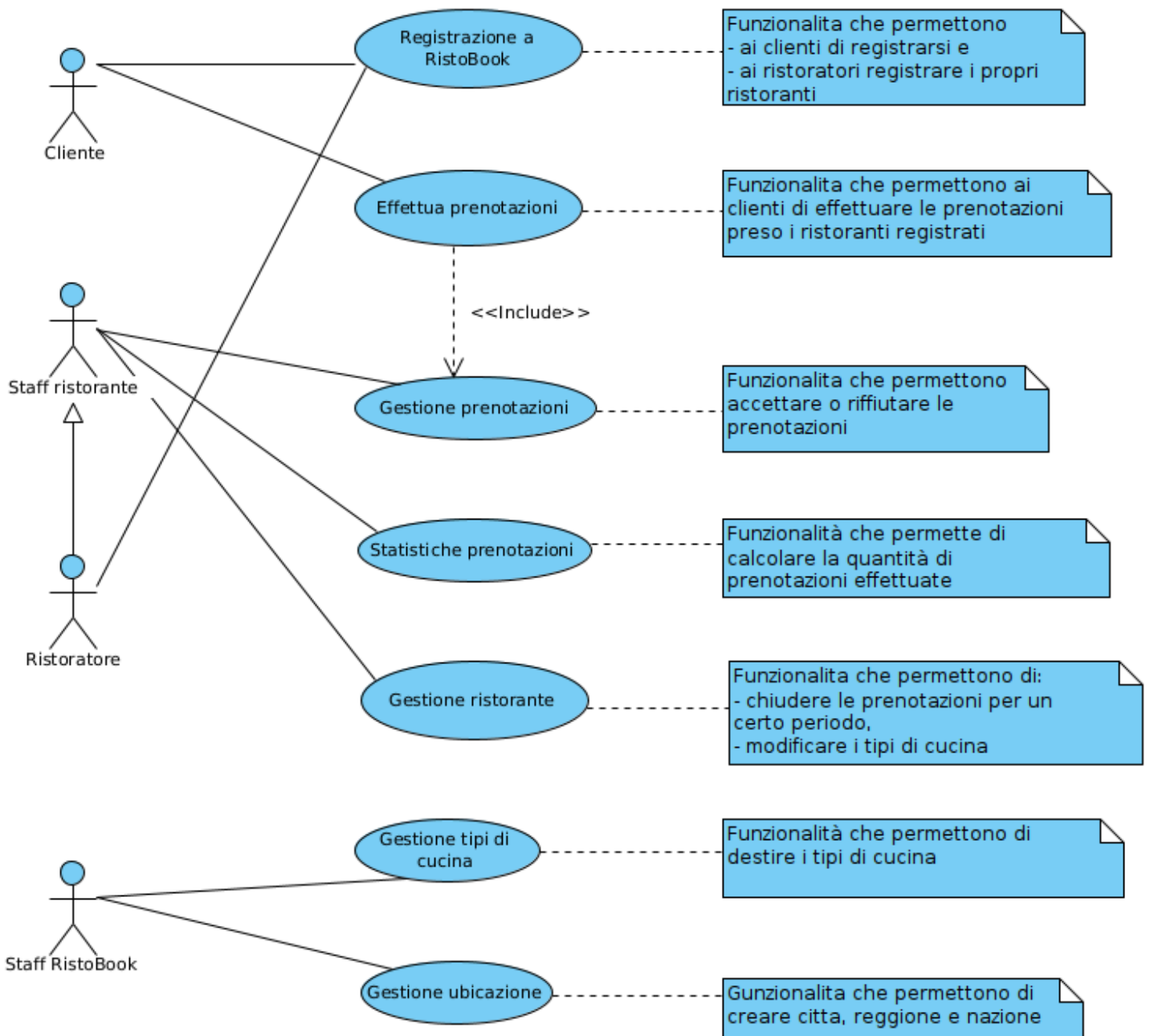
- pre-condizioni:
 - nessuna
- post-condizioni:
 - Operazione non modifica i dati e link (operazione non fa slide-effect)
 - Viene restituito il valore "result" definito come segue
 - Sia L l'insieme dei link (this, pa): cl_pren, pa: Accettata | $d_iniz \leq pa.istante_accettazione \leq d_fine$
 - result: |L|

num_prenot_rifiutate(d_iniz:DataOra, d_fine:DataOra): Intero ≥ 0

ME-approved ▾

- pre-condizioni:
 - nessuna
- post-condizioni:
 - Operazione non modifica i dati e link (operazione non fa slide-effect)
 - Viene restituito il valore "result" definito come segue
 - Sia L l'insieme dei link (this, pr): cl_pren, pr:Rifiutata | $d_iniz \leq pr.istante_rifiuto \leq d_fine$
 - result: |L|

1.4. Diagramma degli Use-Case



###

Separerei meglio 'GestioneRistobook' in un paio di use-case leggermente più specifici. In generale, 'Gestione Sistema' non è uno use-case che aiuta il lettore a capire cosa l'attore può effettivamente fare.

1.5. Specifiche degli Use-Case

1.5.1. Specifica dello use-case Registrazione a RistoBook

Per noi pronto

registrazione_cliente(

nom: Stringa,
cogn: Stringa,
em: Email
) : Cliente

Me-approved

- pre-condizioni:
 - non esiste alcun oggetto di classe *Cliente* con i valori
 - *em* per l'attributo *email*
- post-condizioni:

- viene creato e restituito un nuovo oggetto *result*: *Cliente* con i valori
 - *result.nome* = *nom*
 - *result.cognome* = *cogn*
 - *result.email* = *em*

Per noi **pronto** ▾

bandinelli.lisa@gmail.com

registrazione_ristorante(

nom: Stringa,
ind: Indirizzo,
part: Partita_iva,
P: Periodo[1..*],
cit: Citta,
tip: TipoCucina
) : Ristorante

Me-approved ▾

- pre-condizioni:
 - non esiste alcun oggetto *r* di classe *Ristorante* con il valore *nom* per attributo nome tale che (*r*, *cit*):
ris_cit
- post-condizioni:
 - viene creato e restituito un nuovo oggetto *result*: *Ristorante* con i valori:
 - *result.nome* = *nom*
 - *result.indirizzo* = *ind*
 - *result.partita_iva* = *part*
 - per ogni *pc* in *P*
 - *result.periodo_chiusura* = *pc*
 - viene creato un link (*result*, *cit*): *ris_cit*
 - viene creato un link (*result*, *tip*): *ris_tip*

1.5.2. Specifica dello use-case [Effettua prenotazioni](#)

Per noi **pronto** ▾

effettuare_prenotazioni(

c: Cliente,
r: Ristorante,
num_com: Intero ≥ 0 ,
dat_pren: DataOra
) : Prenotazione

Me-approved ▾

- pre-condizioni:
 - *r.is_in_periodo_chiusura(dat_pren)* = *FALSE*
- post-condizioni:
 - viene creato e restituito un nuovo oggetto *result*: *Prenotazione* con valori
 - *result.istante_creazione* = *istante di adesso*
 - *result.numero_commensali* = *num_com*
 - *result.data_ora_prenotata* = *dat_pren*
 - viene creato un link (*result*, *r*): *pren_ris*
 - viene creato un link (*c*, *result*): *cl_pren*

1.5.3. Specifica dello use-case [Gestione prenotazioni](#)

Per noi **pronto** ▾

ristorante_accetta_prenotazione(*p*: Prenotazione) : Accettata

Me-approved ▾

- pre-condizione:

- p non è istanza né di *Accettata* né di *Rifiutata*
- post-condizioni:
 - modifiche ai dati:
 - p diventa istanza di *Accettata*
 - $result = p$

Per noi pronto ▾

ristorante_rifiuta_prenotazione(p : Prenotazione) : Rifiutata

Me-approved ▾

- pre-condizione:
 - p non è istanza né di *Accettata* né di *Rifiutata*
- post-condizioni:
 - modifiche ai dati:
 - p diventa istanza di *Rifiutata*
 - $result = p$

1.5.4. Specifica dello use-case Gestione ristorante

Per noi pronto ▾

Nataliia Borovyk

serve per aggiungere un nuovo periodo di chiusura

creazione_periodo_di_chiusura(

r : Ristorante,
 d_{iniz} : Data,
 d_{fin} : Data,
 O : OrarioGiorno[1..*])

Me-approved ▾

- pre-condizioni:
 - Per ogni pc : *Periodo* tale che pc è uno dei valori di $r.periodo_chiusura$ (o pc è uno dei valori di $r.periodo_chiusura$)
 - Deve essere vero che
 - $non_sovrapposti_data($

$pc.data_inizio,$
 $pc.data_fine,$
 $d_{iniz},$
 $d_{fin}) = True$
- post-condizioni
 - $result$: viene creato il nuovo valore pc' e aggiunto ai valori $r.periodo_chiusura$

Per noi pronto ▾

Nataliia Borovyk

serve per modificare ora e/o giorno nel periodo di chiusura già esistente

modifica_orario_giorno_in_periodo_chiusura(

r : Ristorante,
 pc : Periodo
 og_nuovo : OrarioGiorno)

Me-approved ▾

- pre-condizioni:
 - esiste un valore pc_1 : Periodo in $r.periodo_chiusura$ tale che $pc_1 = pc$ # versione 1
 o pc è uno dei valori di $r.periodo_chiusura$ # versione 2

- per ogni *og* in *pc.orari*
 - deve essere vero che
 - $non_sovrapposti_orari(giorno(og, og_nuovo) = True$
- post-condizioni:
 - (poiché i valori di un tipo di dato sono immutabili, dovrebbe essere creato un nuovo valore *pc*)
 - Viene creato un nuovo valore *pc₁*: Periodo tale che
 - $pc_1.data_inizio = pc.data_inizio$,
 - $pc_1.data_fine = pc.data_fine$
 - $pc_1.orari = pc.orari \cup \{og_nuovo\}$
 - Viene rimosso *pc* da *r.periodo_chiusura*
 - Viene aggiunto *pc₁* a *r.periodo_chiusura* (sostituisce *pc*)

Per noi **pronto** ▾
 Nataliia Borovyk

elimina_periodo_di_chiusura(

r: Ristorante,
 pc: Periodo)

Me-approved ▾

- pre-condizioni:
 - deve esistere $r.periodo_chiusura = pc$
 - deve esistere almeno un altro valore *pc'*: Periodo per attributo *r.periodo_chiusura* tale che $pc \neq pc'$
- post-condizioni:
 - Viene rimosso *pc* da *r.periodo_chiusura*

Per noi **pronto** ▾

aggiunta_tipo_cucina(r: Ristorante, tip: TipoCucina)

Me-approved ▾

- pre-condizioni:
 - non esiste il link (*r, tip*): *ris_tip*
- post-condizioni:
 - viene creato il link (*r, tip*): *ris_tip*

Per noi **pronto** ▾

elimina_tipo_cucina(r: Ristorante, tip: TipoCucina)

Me-approved ▾

- pre-condizioni:
 - esiste il link (*r, tip*): *ris_tip*
 - deve esistere almeno un altro link (*r, tip'*): *ris_tip*, *tip'*: TipoCucina, $tip \neq tip'$
- post-condizioni:
 - viene eliminato il link (*r, tip*): *ris_tip*

1.5.5. Specifica dello use-case [Statistiche](#)

Per noi **pronto** ▾
 Chiara Macciocchi 😊

num_prenotazioni_ristorante_nel_periodo(

r: Ristorante,
 d_iniz : DataOra,
 d_fine : DataOra
) : Intero ≥ 0

Me-approved ▾

dato un ristorante, calcolare quante prenotazioni ha ricevuto il ristorante in quel periodo

- precondizioni:
d_fine > d_iniz
- postcondizioni:
 - l'operazione non modifica il livello estensionale degli oggetti
 - Il risultato result è così definito:
 - Sia P l'insieme degli oggetti p:Prenotazione tali che
 - esiste il link (r,p):ris_pren AND
 - d_iniz <= p.data_ora_prenotata <= d_fine
 - Sia result il numero di elementi di P (cardinalità di P, |P|)

Per noi **pronto** ▾

Chiara Macciocchi 😊

num_prenot_accettate_ristorante_nel_periodo(

r: Ristorante,
d_iniz : DataOra,
d_fine : DataOra
) : Intero >= 0

Me-approved ▾

dato un ristorante calcolare il numero di prenotazioni accettate da quel ristorante nel periodo indicato

- precondizioni: d_fine > d_iniz
- postcondizioni:
 - l'operazione non modifica il livello estensionale degli oggetti
 - Il risultato result è così definito:
 - Sia P l'insieme degli oggetti p:Accettata tali che
 - esiste il link (r,p):ris_pren AND
 - d_iniz <= p.data_ora_prenotata <= d_fine
 - Sia result il numero di elementi di P (cardinalità di P, |P|)

Per noi **pronto** ▾

Chiara Macciocchi 😊

num_prenot_rifiutate_ristorante_nel_periodo(

r: Ristorante,
d_iniz : DataOra,
d_fine : DataOra
) : Intero >= 0

Me-approved ▾

dato un ristorante calcolare il numero di prenotazioni rifiutate da quel ristorante nel periodo indicato

- precondizioni: d_fine > d_iniz
- postcondizioni:
 - l'operazione non modifica il livello estensionale degli oggetti
 - Il risultato result è così definito:
 - Sia P l'insieme degli oggetti p:Rifiutata tali che
 - esiste il link (r,p):ris_pren AND
 - d_iniz <= p.data_ora_prenotata <= d_fine
 - Sia result il numero di elementi di P (cardinalità di P, |P|)

Per noi **pronto** ▾

Mattia mattocono@gmail.com

Versione modificata

num_prenotazioni_effettuate_da_cliente_nel_periodo(

c: Cliente,
d_iniz : DataOra,
d_fine : DataOra

) : Intero ≥ 0

Me-approved ▾

Dato un cliente c :Cliente il numero di prenotazioni effettuate verso Ristorante e che ha un link r :Ristorante $\in$ $data_inizio \leq periodo \leq data_fine$

- Pre Condizioni:
 - $d_fine \geq d_inizio$
 - Post Condizioni:
 - l'operazione non cambia il livello estensionale dei dati
 - il risultato sia result definito come l'insieme P p :prenotazione tali che
 - esiste un link (c,p) :cl_pren
 - $p.data_inizio \leq p.data_prenotata \leq p.data_fine$
- result = $|P|$

Per noi pronto ▾

Esubalew Grappasonni

num_prenotazioni_effetuate_da_cliente_nel_periodo_nel_ristorante(

c : Cliente,
 r : Ristorante,
 d_iniz : DataOra,
 d_fine : DataOra
) : Intero ≥ 0

Me-approved ▾

Dato un cliente c :Cliente e r :Ristorante, calcolare il numero di prenotazioni effettuate da c :Cliente presso r :Ristorante nel periodo compreso d_iniz , d_fine compreso gli estremi

- Precondizioni:
 - $d_fine > d_inizio$
- Postcondizioni:
 - L'operazione non modifica il livello estensionale degli oggetti
Il risultato result è definito come segue:
 - Sia P l'insieme degli oggetti p : Prenotazione tali che:
 - esiste il link (c, p) : cli_pren
 - esiste il link (r, p) : ris_pren (la prenotazione p è destinata al ristorante r)
 - $d_iniz \leq p.data_ora_prenotata \leq d_fine$
 - result = $|P|$ (cardinalità dell'insieme P , cioè il numero di prenotazioni trovate)

1.5.6. Specifica dello use-case Gestione Ubicazione

Per noi pronto ▾

Lisa Bandinelli

creazione_citta(nome: Stringa, reg: Regione) : Citta

Me-approved ▾

- pre-condizioni:
 - non esiste c :Citta tale che $c.nome = nome$ e (c, reg) :cit_reg
- post-condizioni:
 - viene creata un'istanza di c :Citta con $c.nome = nome$;
 - viene creato il link (c, reg) : cit_reg

Per noi pronto ▾

Lisa Bandinelli

creazione_regione(nome: Stringa, naz: Nazione) : Regione

Me-approved ▾

- pre-condizioni:
 - non esiste r:Regione tale che r.nome = nome e (r, naz): reg_naz
- post-condizioni:
 - viene creata un'istanza di r:Regione con c.nome = nome;
 - viene creato il lin (r, naz): reg_naz

Per noi **pronto** ▾

cecilembazoa4@gm...

creazione_nazione(nome: Stringa) : Nazione

Me-approved ▾

- precondizioni:
 - non deve esistere un oggetto n:Nazione(un oggetto n di classe Nazione) con il valore per attribuito nome uguale a " name" ,(n.nome = name)
- Postcondizioni:
 - il risultato result è così definito:
 - viene creato e restituito un nuovo oggetto n:Nazione con un valore per attribuito nome uguale a "name" (n.nazione = name)

1.5.7. Specifica dello use-case Gestione tipi cucina

Per noi **pronto** ▾

cecilembazoa4@gmail.com

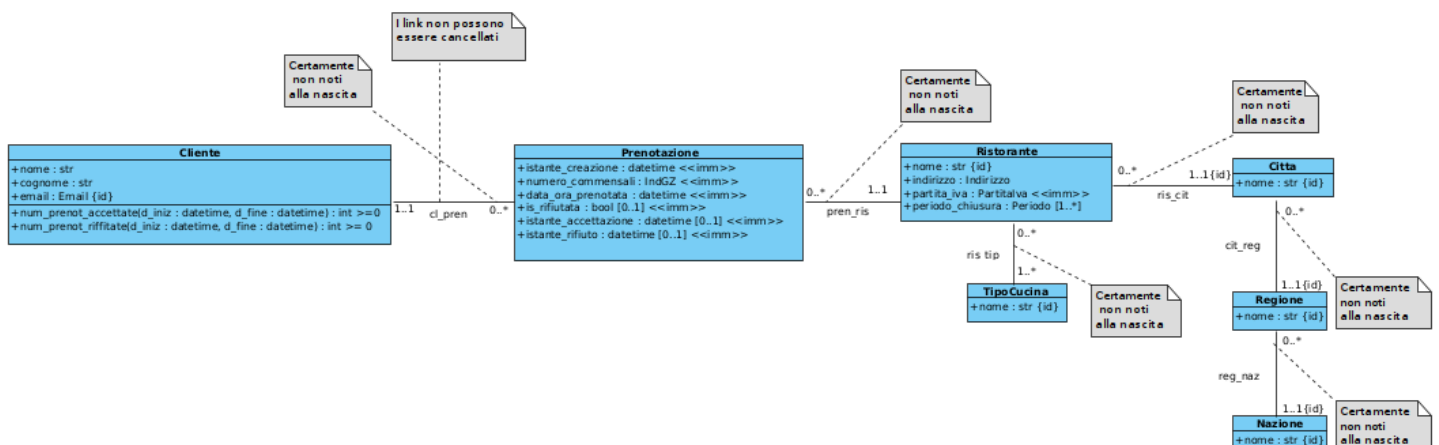
creazione_tipo_cucina(nom: Stringa) : TipoCucina

Me-approved ▾

- precondizioni:
 - non deve esistere un oggetto tp: TipoCucina (un oggetto tp di classe TipoCucina) con il valore per attribuito nome uguale a "nom" (tp.nome = nom)
- postcondizioni:
 - il risultato result è così definito :
 - viene Creato e restituito un nuovo oggetto tc: TipoCucina con un valore per attribuito nome uguale a "nom" (tc.nome = nom)

2. Design dell'applicazione Python

2.1. Diagramma UML delle classi Ristrutturato



2.2. Specifiche delle Classi

2.2.1. Specifica della classe Prenotazione

Operazioni Ausiliarie

non_sovrapposti(i_1 : DataOra, f_1 : DataOra, i_2 : DataOra, f_2 : DataOra) : Booleano

- precondizioni:
 - $i_1 \leq f_1$ e $i_2 \leq f_2$
- postcondizioni:
 - result = True se e solo se:
 - $i_1 > f_2$ or $f_1 < i_2$

Per noi **pronto** ▾

Chiara Macciocchi

N1

non_sovrapposti(i_1 : datetime, f_1 : datetime, i_2 : datetime, f_2 : datetime) : bool

ME-approved ▾

algoritmo:

```
if i1 > f1 OR i2 > f2:
    ERRORE
IF i1 > f2 OR i2 > f1:
    return TRUE
altrimenti:
    return FALSE
```

non_sovrapposti_data(i_1 : Data, f_1 : Data, i_2 : Data, f_2 : Data) : Booleano

- precondizioni:
 - $i_1 \leq f_1$ e $i_2 \leq f_2$
- postcondizioni:
 - result = True se e solo se:
 - $i_1 > f_2$ or $f_1 < i_2$

Per noi **pronto** ▾

Chiara Macciocchi

N2

non_sovrapposti_data(i_1 :datetime.date, f_1 :datetime.date, i_2 :datetime.date, f_2 : datetime.date): bool

ME-approved ▾

algoritmo:

```
if i1 > f1 OR i2 > f2:
    ERRORE
IF i1 > f2 OR i2 > f1:
    return TRUE
altrimenti:
    return FALSE
```

non_sovrapposti_orariogiorno(og1: OrarioGiorno, og2: OrarioGiorno): Booleano

- precondizioni:
 - nessuna
- postcondizioni:

```

○ Result = True
  ■ se e solo se
    • [ og1.giorno_settimana ≠ og2.giorno_settimana
      oppure
      • og1.giorno_settimana = og2.giorno_settimana
        e
        • NON ESISTE un t: Ora tale che
          ○ og1.ora_inizio ≤ t ≤ og1.ora_fine e
          ○ og2.ora_inizio ≤ t ≤ og2.ora_fine ]

```

Per noi **pronto** ▾
Nataliia Borovyk

N3

non_sovrapposti_orariogiorno(og1: OrarioGiorno, og2: OrarioGiorno): bool

ME-approved ▾

algoritmo:

```

se og1.giorno_settimana != og2.giorno_settimana
    return True
se og2.ora_fine <= og1.ora_inizio or og1.ora_fine <= og2.ora_inizio
    return True
altrimenti
    return False

```

Vincoli Esterni

[V.IstanteCreazioneMinoreDataOraPrenotata]

ME-approved ▾

Per ogni p:Prenotazione deve essere vero che p.istante_creazione < p.data_ora_prenotata

[V.IstanteCreazioneMinoreDiAccettazione]

ME-approved ▾

Per ogni p:Prenotazione deve essere vero che p.istante_creazione < p.istante_accettazione

[V.IstanteCreazioneMinoreDiRifiuto]

ME-approved ▾

Per ogni p:Prenotazione deve essere vero che p.istante_creazione < p.istante_rifiuto

[V.DataOraPrenotataNonInPeriodoChiusura]

ME-approved ▾

Per ogni p:Prenotazione con valore i = p.data_ora_prenotata e (p,r): pren_ris, r:Ristorante.
 r.is_in_periodo_chiusura(i) = FALSE

[Da fusione] per ogni p:Prenotazione:

ME-approved ▾

- p.istante_accettazione non è valorizzato oppure p.istante_rifiuto non è valorizzato
- p.is_rifiutata non è valorizzato se e solo se p.istante_accettazione e p.istante_rifiuto non sono valorizzati
errore
- p.is_accettata = TRUE se e solo se p.istante_accettazione è valorizzato;

- p.is_rifiutata = TRUE se e solo se p.istante_rifiuto è valorizzato;
- p.is_accettata non è valorizzato and p.is_rifiutata non è valorizzato se e solo se p.istante_accettazione non è valorizzato and p.istante_rifiuto non è valorizzato

[v.OrarioGiorno.NonSovrapposto]

ME-approved ▾

- Per ogni r:Ristorante
 - Per ogni pc:Periodo tale che pc è uno dei valori di r.periodo_chiusura;
 - Per ogni coppia di valori distinti og1:OrarioGiorno, og2:OrarioGiorno di pc.orari, deve essere vero che
 - non_sovrapposti_orariogiorno(
 - og1:OrarioGiorno,
 - og2:OrarioGiorno) = True

[v.OrarioGiorno.NonSovrapposto]

ME-approved ▾

- Per ogni r:Ristorante
 - Per ogni pc:Periodo tale che pc è uno dei valori di r.periodo_chiusura;
 - Per ogni coppia di valori distinti og1:OrarioGiorno, og2:OrarioGiorno di pc.orari, deve essere vero che
 - non_sovrapposti_orariogiorno(
 - og1:OrarioGiorno,
 - og2:OrarioGiorno) = True

2.2.2. Specifica della classe Ristorante

Operazioni ausiliarie

is_in_periodo_chiusura(i: DataOra) : Booleano

- precondizioni : nessuna
- postcondizioni:
 - result = FALSE ⇔ [
 - i ∉ [this.periodo_chiusura.data_inizio, this.periodo_chiusura.data_fine]
 - oppure
 - per ogni valore og di this.periodo_chiusura.orari
 - i.data.giorno.settimana ≠ og.giorno_settimana
 - oppure
 - (i.data.ora <= og.ora_inizio) or i.data.ora >= og.ora_fine]

Per noi pronto ▾
Nataliia Borovyk

N6

is_in_periodo_chiusura(i: datetime) : bool

ME-approved ▾

algoritmo:

```

for pc in this.periodo_chiusura
  If i.data >= pc.periodo_chiusura.data_inizio and i.data <= pc.periodo_chiusura.data_fine:
    for og in pc.orari
      if i.data.giorno_settimana == og.giorno_settimana
        and
        i.ora >= og.ora_inizio and i.ora < og.ora_fine
          return True
return False

```

2.2.3. Specifiche di classe Cliente

num_prenot_accettate(d_iniz:DataOra, d_fine:DataOra): Intero >=0

- pre-condizioni:
 - nessuna
- post-condizioni:
 - Operazione non modifica i dati e link (operazione non fa slide-effect)
 - Viene restituito il valore “result” definito come segue
 - Sia L l’insieme dei link (this, pa): cl_pren, pa: Accettata | d_iniz <= pa.istante_accettazione <= d_fine
 - result: |L|

Per noi **da ricontrollare insieme** ▾

Chiara Macciocchi

N7

num_prenot_accettate(d_iniz: datetime, d_fine: datetime): IntGEZ

Da revisionare ▾

algoritmo:

```
se |this.cl_pren| == 0: return 0
altrimenti:
    n = 0
    for each l in this.cl_pren DO
        IF l.prenotazione.is_accettata == True
            n = n + 1
    return n
```

num_prenot_rifiutate(d_iniz:DataOra, d_fine:DataOra): Intero >=0

- pre-condizioni:
 - nessuna
- post-condizioni:
 - Operazione non modifica i dati e link (operazione non fa slide-effect)
 - Viene restituito il valore “result” definito come segue
 - Sia L l’insieme dei link (this, pr): cl_pren, pr:Rifiutata | d_iniz <= pr.istante_rifiuto <= d_fine
 - result: |L|

Per noi **da ricontrollare insieme** ▾

Chiara Macciocchi

N8

num_prenot_rifiutate(d_iniz: datetime, d_fine: datetime): IntGEZ

Da revisionare ▾

algoritmo:

```
se |this.cl_pren| = 0 return 0
altrimenti:
    n = 0
    for each l in this.cl_pren DO
        IF l.prenotazione == False
            n = n + 1
    return n
```

2.3. Specifiche degli Use-Case

2.3.1. Specifica dello use-case [Registrazione a RistoBook](#)

```
registrazione_cliente(  
    nom: Stringa,  
    cogn: Stringa,  
    em: Email  
    ) : Cliente  
  
● pre-condizioni:  
    ○ non esiste alcun oggetto di classe Cliente con i valori  
        ■ em per l'attributo email  
  
● post-condizioni:  
    ○ viene creato e restituito un nuovo oggetto result: Cliente con i valori  
        ■ result.nome = nom  
        ■ result.cognome = cogn  
        ■ result.email = em
```

Per noi [da ricontrollare insieme](#) **Esubalew Grappasonni**

N9

```
registrazione_cliente(  
    nom: str,  
    cogn: str,  
    em: Email  
    ) : Cliente | None
```

[Da revisionare](#)

algoritmo :

Controllare se esiste un *c* nell'insieme *L* [*c:Cliente*] con la stessa *em:Email*
per ogni *c:Cliente* in *L*:
 se *c.email==em*
 return None

Crea nuovo *c:Cliente* con
 c.nome=nom
 c.cognome=cogn
 c.email=em

Aggiungi *c:Clienti* in *L*

return *c:Cliente*

```
registrazione_ristorante(  
    nom: Stringa,  
    ind: Indirizzo,  
    part: Partita_iva,  
    P: Periodo[1..*],  
    cit: Citta,  
    tip: TipoCucina  
    ) : Ristorante  
  
● pre-condizioni:  
    ○ non esiste alcun oggetto r di classe Ristorante con il valore nom per attributo nome tale che (r, cit): ris_cit  
  
● post-condizioni:  
    ○ viene creato e restituito un nuovo oggetto result: Ristorante con i valori:  
        ■ result.nome = nom  
        ■ result.indirizzo = ind  
        ■ result.partita_iva = part
```

- per ogni *pc* in *P*
 - *result.periodo_chiusura* = *pc*
- viene creato un link (*result*, *cit*): *ris_cit*
- viene creato un link (*result*, *tip*): *ris_tip*

Per noi pronto ▾

Mattia Cona

N10

registrazione_ristorante(

nom: str,
ind: Indirizzo,
part: Partita_iva,
P: Periodo[1..*],
cit: Citta,
tip: TipoCucina
) : Ristorante

Da revisionare ▾

algoritmo:

controllo se esiste il ristorante

se già esiste un ristorante *r* con *r.nome*==*nom* and *r.ris_cit.città*==*cit*
return errore

altrimenti:

creo il ristorante
nome=*nom*
indirizzo=*ind*
p iva=*p iva*
r.periodo chiusura=*pc*
aggiungo alla lista il nuovo ristorante
return nuovo ristorante

2.3.2. Specifica dello use-case [Effettua prenotazioni](#)

effettuare_prenotazioni(

c: Cliente,
r: Ristorante,
num_com: Intero >= 0,
dat_pren: DataOra
) : Prenotazione

- pre-condizioni:
 - *r.is_in_periodo_chiusura(dat_pren)* = FALSE
- post-condizioni:
 - viene creato e restituito un nuovo oggetto *result*: Prenotazione con valori
 - *result.istante_creazione* = istante di adesso
 - *result.numero_commensali* = *num_com*
 - *result.data_ora_prenotata* = *dat_pren*
 - viene creato un link (*result*, *r*): *pren_ris*
 - viene creato un link (*c*, *result*): *cl_pren*

Per noi da ricontrollare insieme ▾

Nataliia Borovyk

N11

effettuare_prenotazioni(

c: Cliente,
r: Ristorante,
num_com: IntGEZ ,

dat_pren: datetime
) : Prenotazione

Da revisionare ▾

algoritmo:

```
se r.is_in_periodo_chiusura(dat_pren):  
    return errore  
viene creato p : Prenotazione tale che  
    p.istante_creazione = adesso,  
    p.numero_commensali = num_com,  
    p.data_ora_prenotata = dat_pren)  
viene creato link (p, r): pren_ris  
viene creato link (c, p): cl_pren  
return p
```

2.3.3. Specifica dello use-case Gestione prenotazioni

ristorante_accetta_prenotazione(p: Prenotazione) : Accettata

- pre-condizione:
 - *p non è istanza né di Accettata né di Rifiutata*
- post-condizioni:
 - modifiche ai dati:
 - *p diventa istanza di Accettata*
 - *result = p*

Per noi da ricontrollare insieme ▾

Chiara Macciocchi

N12

ristorante_accetta_prenotazione(p: Prenotazione)

Da revisionare ▾

algoritmo:

```
if p.is_rifiutata è valorizzato:  
    ERRORE  
altrimenti:  
  
    p.istante_accettazione = adesso  
    p.is_rifiutata = True
```

ristorante_rifiuta_prenotazione(p: Prenotazione) : Rifiutata

- pre-condizione:
 - *p non è istanza né di Accettata né di Rifiutata*
- post-condizioni:
 - modifiche ai dati:
 - *p diventa istanza di Rifiutata*
 - *result = p*

Per noi da ricontrollare insieme ▾

Cecile Mbazoa

N13

ristorante_rifiuta_prenotazione(p: Prenotazione)

Da revisionare ▾

algoritmo:

versione 2

IF p.is_rifiutata è valorizzato: ERRORE
ERRORE

altrimenti:

p.istante_rifiuto = adesso

p.is_rifiutata = TRUE

2.3.4. Specifica dello use-case Gestione ristorante concettuale

creazione_periodo_di_chiusura(

r: Ristorante,

d_iniz: Data,

d_fin: Data,

O: OrarioGiorno[1..*])

• pre-condizioni:

○ Per ogni *pc*: *Periodo* tale che *pc* è uno dei valori di *r.periodo_chiusura*
(alternativa: *pc* è uno dei valori di *r.periodo_chiusura*)

■ Deve essere vero che

• *non_sovrapposti_data*(

pc.data_inizio,

pc.data_fine,

d_iniz,

d_fin) = True

• post-condizioni

○ result: viene creato il nuovo valore *pc'* e aggiunto ai valori *r.periodo_chiusura*

Per noi da ricontrollare insieme ▾

Nataliia Borovyk

N14

creazione_periodo_di_chiusura(

r: Ristorante,

d_iniz: datetime.date,

d_fin: datetime.date,

O: OrarioGiorno[1..*]

)

Da revisionare ▾

algoritmo:

per *pc* in *r.periodo_chiusura*:

se NOT *non_sovrapposti_data*(*pc.data_inizio*, *pc.data_fine*, *d_iniz*, *d_fin*)

result errore

viene creato *pc'*: *Periodo* tale che

pc'.data_inizio = *d_iniz*

pc'.data_fine = *d_fin*

for each *og* in *O*

viene aggiunto *og* in *pc'.orari*

viene aggiunto *pc'* nel *r.periodo_chiusura*

modifica_orario_giorno_in_periodo_chiusura(

r: Ristorante,
pc: Periodo
og_nuovo: OrarioGiorno)

- pre-condizioni:
 - esiste un valore pc_1 : Periodo in *r.periodo_chiusura* tale che $pc_1 = pc$
(alternativa: *pc* è uno dei valori di *r.periodo_chiusura*)
 - per ogni *og* in *pc.orari*
 - deve essere vero che
 - *non_sovraposti_orariogiorno(og, og_nuovo) = True*
- post-condizioni:
 - (poiché i valori di un tipo di dato sono immutabili, dovrebbe essere
 - Viene creato un nuovo valore pc_1 : Periodo tale che
 - $pc_1.data_inizio = pc.data_inizio$,
 - $pc_1.data_fine = pc.data_fine$
 - $pc_1.orari = pc.orari \cup \{og_nuovo\}$
 - Viene rimosso *pc* da *r.periodo_chiusura*
 - Viene aggiunto pc_1 a *r.periodo_chiusura* (sostituisce *pc*)

Per noi da ricontrollare insieme ▾

Nataliia Borovyk

N15

modifica_orario_giorno_in_periodo_chiusura(

r: Ristorante,
pc: Periodo,
og_nuovo: OrarioGiorno)

Da revisionare ▾

algoritmo:

```
se pc NOT in r.periodo_chiusura:
    result errore
per ogni og in pc.orari:
    se NOT non_sovraposti_orariogiorno(og, og_nuovo):
        result errore
nuovi_orari = coppia pc.orari
nuovi_orari.add(og_nuovo)
viene creato pc1: Periodo tale che
    pc1.data_inizio = pc.data_inizio,
    pc1.data_fine = pc.data_fine
    pc1.orari = nuovi_orari
viene rimosso pc da r.periodo_chiusura
viene aggiunto pc1 a r.periodo_chiusura
```

elimina_periodo_di_chiusura(

r: Ristorante,
pc: Periodo)

- pre-condizioni:
 - deve esistere *r.periodo_chiusura = pc*
 - deve esistere almeno un altro valore pc' : Periodo per attributo *r.periodo_chiusura* tale che $pc \neq pc'$

- post-condizioni:
 - Viene rimosso pc da r.periodo_chiusura

Per noi **pronto** ▾

Mattia Cona

N16

elimina_periodo_di_chiusura(

r: Ristorante,
pc: Periodo)

Da revisionare ▾

algoritmo:

```
se |r.periodo_chiusura| >=2
    per ogni pc' in r.periodo_chiusura:
        se pc=pc'
            rimuovo pc'
            break
```

ritorna: errore

aggiunta_tipo_cucina(r: Ristorante, tip: TipoCucina)

- pre-condizioni:
 - non esiste il link $(r, tip): ris_tip$
- post-condizioni:
 - viene creato il link $(r, tip): ris_tip$

Per noi **da ricontrollare insieme** ▾

Lisa Bandinelli

N17

aggiunta_tipo_cucina(r: Ristorante, tip: TipoCucina)

Da revisionare ▾

algoritmo:

```
for tc in Tipocucina:
    se (r, tip) in tip_cuc: return ERRORE
tc.nome = nome
crea il link (tc, r): tip_cuc:
    result = tip_cuc
```

elimina_tipo_cucina(r: Ristorante, tip: TipoCucina)

- pre-condizioni:
 - esiste il link $(r, tip): ris_tip$
 - deve esistere almeno un altro link $(r, tip'): ris_tip, tip': TipoCucina, tip \neq tip'$
- post-condizioni:
 - viene eliminato il link $(r, tip): ris_tip$

Per noi **da ricontrollare insieme** ▾

Esubalew Grappasonni

N18

elimina_tipo_cucina(r: Ristorante, tip: TipoCucina)

Da revisionare ▾

algoritmo:

```

Per ogni coppia (r,tip) in TipoCucina:
  se (r,tip) not in TipoCucina:
    return None
elimina_tipo_cucina(r,tip)

```

2.3.5. Specifica dello use-case Statistiche

```

num_prenotazioni_ristorante_nel_periodo(
    r: Ristorante,
    d_iniz : DataOra,
    d_fine : DataOra
) : Intero >= 0

```

dato un ristorante, calcolare quante prenotazioni ha ricevuto il ristorante in quel periodo

- precondizioni:
 - d_fine > d_iniz
- postcondizioni:
 - l'operazione non modifica il livello estensionale degli oggetti
 - Il risultato result è così definito:
 - Sia P l'insieme degli oggetti p:Prenotazione tali che
 - esiste il link (r,p):ris_pren AND
 - d_iniz <= p.data_ora_prenotata <= d_fine
 - Sia result il numero di elementi di P (cardinalità di P, |P|)

Per noi da ricontrollare insieme ▾

Nicolás VR

N19

```

num_prenotazioni_ristorante_nel_periodo(
    r: Ristorante,
    d_iniz : datetime,
    d_fine : datetime
) : IntGEZ

```

Da revisionare ▾

algoritmo:

```

se d_fine <= d_iniz:
    return ERRORE intervallo non valido

conteggio = 0
per ogni p in r.ris_pren:
    se d_iniz <= p.data_ora_prenotata <= d_fine:
        conteggio = conteggio + 1
return conteggio

```

```

num_prenot_accettate_ristorante_nel_periodo(
    r: Ristorante,
    d_iniz : DataOra,
    d_fine : DataOra
) : Intero >= 0

```

dato un ristorante calcolare il numero di prenotazioni accettate da quel ristorante nel periodo indicato

- precondizioni: d_fine > d_iniz
- postcondizioni:
 - l'operazione non modifica il livello estensionale degli oggetti

- Per noi **da ricontrollare insieme** ▾
- Nicolás VR**

N20

```
num_prenot_accettate_ristorante_nel_periodo(
    r: Ristorante,
    d_iniz : datetime,
    d_fine : datetime
) : IntGEZ
```

Da revisionare ▾

algoritmo:

```

se d_fine <= d_iniz:
    return ERRORE "intervallo non valido"

conteggio = 0
per ogni p in r.ris_pren:
    se d_iniz <= p.data_ora_prenotata <= d_fine and p.accettato = True:
        conteggio = conteggio + 1

return conteggio

```

```
num_prenot_rifiutate_ristorante_nel_periodo(
    r: Ristorante,
    d_iniz : DataOra,
    d_fine : DataOra
) : Intero >= 0
```

dato un ristorante calcolare il numero di prenotazioni rifiutate da quel ristorante nel periodo indicato

- precondizioni: $d_fine > d_iniz$
- postcondizioni:
 - l'operazione non modifica il livello estensionale degli oggetti
 - Il risultato result è così definito:
 - Sia P l'insieme degli oggetti p: Rifiutata tali che
 - esiste il link (r,p):ris_pren AND
 - $d_iniz \leq p.data_ora_prenotata \leq d_fine$
 - Sia result il numero di elementi di P (cardinalità di P, |P|)

Per noi **da ricontrrollare insieme**

Esubalew Grappasonni

N21

```
num_prenot_rifiutate_ristorante_nel_periodo(
    r: Ristorante,
    d_iniz : datetime,
    d_fine : datetime
) : IntGEZ
```

Da revisionare ▾

algoritmo:

```
se d_inizio <= d_fine
    return ERRORE
```

n rifiutati = 0

```

per ogni p:Rifiutata in P:
    se d_iniz <= p.data_ora_prenotata <= d_fine
        n_rifiutati += 1
return n_rifiutati

```

```

num_prenotazioni_effettuate_da_cliente_nel_periodo(
    c: Cliente,
    d_iniz : DataOra,
    d_fine : DataOra
) : Intero >=0

```

Dato un cliente c:Cliente il numero di prenotazioni effettuate verso Ristorante e che ha un link r:Ristorante $\in$ data_inizio<=periodo<=data_fine

- Pre Condizioni:
 - d_fine >= d_inizio
 - Post Condizioni:
 - l'operazione non cambia il livello estensionale dei dati
 - il risultato sia result definito come l'insieme P p:prenotazione tali che
 - esiste un link (c,p):cl_pren
 - p.data_inizio <=p.data_prenotata <=p.data_fine
- result = |P|

Per noi [da ricontrollare insieme](#) ▾

Lisa Bandinelli

N22

[Da revisionare](#) ▾

algoritmo:

```

num_prenotazioni_effettuate_da_cliente_nel_periodo(
    c: Cliente,
    d_iniz: Datetime,
    d_fine: Datetime
): IntGEZ

se d_fine <= d_inizio:
    return ERRORE
altrimenti:
    PE = {}
per ogni l in c.cl_pren:
    se d_iniz <= l.prenotazione.istante_creazione <= d_fine:
        PE.add(l)
se |PE| == 0: result None
return |PE|

```

```

num_prenotazioni_effettuate_da_cliente_nel_periodo_nel_ristorante(
    c: Cliente,
    r: Ristorante,
    d_iniz : DataOra,
    d_fine : DataOra
) : Intero >=0

```

Dato un cliente c:Cliente e r:Ristorante, calcolare il numero di prenotazioni effettuate da c:Cliente presso r:Ristorante nel periodo compreso d_iniz, d_fine compreso gli estremi

- Precondizioni:

- `d_fine > d_inizio`
- Postcondizioni:
 - L'operazione non modifica il livello estensionale degli oggetti
 - Il risultato `result` è definito come segue:
 - Sia P l'insieme degli oggetti p : Prenotazione tali che:
 - esiste il link (c, p) : `cli_pren`
 - esiste il link (r, p) : `ris_pren` (la prenotazione p è destinata al ristorante r)
 - $d_{iniz} \leq p.data_ora_prenotata \leq d_{fine}$
 - `result = |P|` (cardinalità dell'insieme P , cioè il numero di prenotazioni trovate)

Per noi da ricontrollare insieme ▾

Mattia Cona

N23

num_prenotazioni_effettuate_da_cliente_nel_periodo_nel_ristorante(

c: Cliente,
r: Ristorante,
d_iniz : datetime,
d_fine : datetime
) : IntGEZ

Da terminare ▾

algoritmo:

```

se d_fine <= d_inizio
    ritorna errore

p = []
p per ogni p in c.cli_pre:
    se p.risto = r
        (aggiungere)

        p.data_ora_prenotata compresa tra d_inizio e d_fine)
        result = numero elementi della lista p
    ritorna result
  
```

2.3.6. Specifica dello use-case Gestione Ubicazione

creazione_citta(nome: Stringa, reg: Regione) : Citta

- pre-condizioni:
 - non esiste c :Citta tale che $c.nome = nome$ e $(c, reg):cit_reg$
- post-condizioni:
 - viene creata un'istanza di c :Citta con $c.nome = nome$;
 - viene creato il link $(c, reg): cit_reg$

Per noi da ricontrollare insieme ▾

Cecile Mbazoa

N24

creazione_citta(nome: str, reg: Regione) : Citta

Da revisionare ▾

algoritmo:

```

for c in citta:
    se c.nom == nome and (c,reg) in cit_reg:
        ERRORE

crea nuova_citta = Citta()
  
```

```

nuova_citta.nome = nome
nuova_citta.aggiungo_citta
crea (nuova_citta, reg) : cit_reg
    result = nuova_citta

```

creazione_regione(nome: Stringa, naz: Nazione) : Regione

- pre-condizioni:
 - non esiste r:Regione tale che r.nome = nome e (r, naz): reg_naz
- post-condizioni:
 - viene creata un'istanza di r:Regione con c.nome = nome;
 - viene creato il lin (r, naz): reg_naz

Per noi [da ricontrollare insieme](#) ▾

Lisa Bandinelli

N25

creazione_regione(nome: str, naz: Nazione) : Regione

[Da revisionare](#) ▾

algoritmo:

```

for r in Regione:
    se r.nome == nome and (r, naz) in reg_naz:
        return ERRORE
r = Regione()
r.nome = nome
aggiungi (r, naz) a reg_naz:
    result = r

```

creazione_nazione(nome: Stringa) : Nazione

- precondizioni:
 - non deve esistere un oggetto n:Nazione(un oggetto n di classe Nazione) con il valore per attribuito nome uguale a " name" ,(n.nome = name)
- Postcondizioni:
 - il risultato result è così definito:
 - viene creato e restituito un nuovo oggetto n:Nazione con un valore per attribuito nome uguale a "name" (n.nazione = name)

Per noi [da ricontrollare insieme](#) ▾

Nicolás VR

N26

creazione_nazione(nome: str) : Nazione

[Da revisionare](#) ▾

algoritmo:

```

nazioni = insieme di oggetti Nazione
per ogni n in nazioni:
    se n.nome == nome:
        return ERRORE la nazione esiste già, non si crea nulla
n = nuova Nazione()
n.nome = nome
return n

```

2.3.7. Specifica dello use-case [Gestione tipi cucina](#)

creazione_tipo_cucina(nom: Stringa) : TipoCucina

- precondizioni:
 - non deve esistere un oggetto tp: TipoCucina (un oggetto tp di classe TipoCucina) con il valore per attributo nome uguale a “nom” (tp.nome = nom)
- postcondizioni:
 - il risultato result è così definito :
 - viene Creato e restituito un nuovo oggetto tc: TipoCucina con un valore per attributo nome uguale a “nom” (tc.nome = nom)

Per noi [da ricontrollare insieme](#) ▾

Cecile Mbazona

N27

creazione_tipo_cucina(nom: str) : TipoCucina

[Da revisionare](#) ▾

algoritmo:

```
tipi_cucina = {}
for tp i tipi_cucina:
    se tp.nome == nome:
        return ERRORE
crea tc =TipoCucina()
tc.nome = nome
aggiungi tc in tipi_cucina
return tc
```